

1. USART 和 UART 的区别？

答:1.同步与异步通信：最主要的区别在于同步和异步通信的支持。**USART** 支持同步（**Synchronous**）和异步（**Asynchronous**）两种通信模式，而 **UART** 仅支持异步通信模式。

2.时钟信号：**USART** 在同步通信模式下需要一个外部的时钟信号来同步数据传输，而在异步通信模式下不需要时钟信号。**UART** 只能通过异步方式进行数据传输，因此不需要时钟信号。

3.数据帧格式：**USART** 提供了更多的数据帧格式选项，包括数据位数、停止位数、奇偶校验等，并且可以通过配置来实现更高的数据传输速率。**UART** 则通常仅支持常见的数据帧格式，如 8 个数据位、1 个停止位。

4.错误检测：在 **USART** 中，由于同步通信的支持，可以进行更强大的错误检测，如使用差错检测码（**CRC**）来保证数据的完整性。而在 **UART** 中，由于仅支持异步通信，错误检测的能力相对较弱。

5.**USART** 和 **UART** 在硬件电路层面上可能会存在些许差异，但它们的基本功能都是提供串行通信的能力。在实际应用中，选择 **USART** 还是 **UART** 取决于具体需求和硬件支持情况。

2. UART 优缺点？

答：优点： 简单易用：**UART** 是一种简单的串行通信接口，容易实现和使用。它只需要两条信号线（发送线和接收线），不需要复杂的时钟同步机制，适合于嵌入式系统和普通计算设备。

广泛兼容性：**UART** 是一种标准接口，广泛用于各种设备之间的串行通信，例如计算机与外部设备（如传感器、显示屏、**GPS** 模块等）之间的数据交互。

低成本：由于 **UART** 的简单性，相对于其他复杂的串行通信接口，它的硬件成本较低，使得它在成本敏感的应用中非常有吸引力。

通信距离：**UART** 的异步传输方式使得它在通信距离上具有一定的优势。它可以通过电平转换器和适当的线缆来实现长距离的通信。

缺点：速度限制：由于 **UART** 是异步通信，其传输速度受到一些限制。较高的数据传输速率可能会导致数据精度下降或信号失真。相比之下，同步通信接口（如 **SPI** 和 **I2C**）在高速数据传输方面更有优势。

无错恢复机制：**UART** 缺乏强大的错误检测和纠正机制。它没有内置的差错检测码（如 **CRC**），因此对于数据完整性的要求较高的应用，**UART** 可能不太适合。

只支持点对点通信：**UART** 是一种点对点通信协议，每个 **UART** 接口只能实现两个设备之间的直接通信。如果需要多个设备进行通信，可能需要额外的复用器或其他解决方案。

3. IIC 三种信号

答：时钟信号，数据信号、地址

4. 画 IIC 时序图

5. 数据传输流程

6. IIC 工作速率

答：标准模式、快速模式、高速模式

7. 怎样用 IO 口模拟 IIC?

答：IO 口的设置通常为开漏输出模式和开漏输入模式，并配合外部上拉电阻的使用。开漏输出模式下，IO 口可以输出低电平或高阻态（高电平时，IO 口处于三态，即不输出电平）。这样可以与其他设备的开漏输入端口连接，形成一个共享的总线结构，IO 口输出对应的时序即可进行通信

8. IIC 如何进行读写操作?

答：七位地址位后设置 0/1 进行设置读写

9. IIC 总线仲裁机制是怎么样的?

答：当多人设备同时请求访问总线时，主设备会发送一个仲裁信号，这个信号包含一个特殊的比特位，称为“仲裁位”。所有设备都会监听这个仲裁位，如果发现自己发送的地址与仲裁信号中的地址不匹配，就会放弃访问总线，等待下一次机会

10. IIC 电气特性，为什么要使用上拉电阻，IIC 有哪些模式，传输速率如何。IIC 传输中可能会遇到哪些错误?

答：使用上拉电阻的目的是确保在 SDA 和 SCL 线上的电平可以稳定地保持在高电平状态。当总线上没有任何设备向下拉低 SDA 线时，上拉电阻将提供一个足够的电流，使得 SDA 线上的电压保持在高电平。

标准模式（时钟频率最高为 100 kHz）、快速模式（最高可达 400 kHz）、高速模式（最高可达 3.4 MHz）

总线冲突、应答错误、超时、地址错误、校验错误

11. 介绍 SPI

答：我主要接触的 spi 有两种，一个三线 spi、一个四线 spi，有时钟信号 SCLK、从设备输入 MOSI、主设备输入 MISO、片选 CS，支持全双工通信

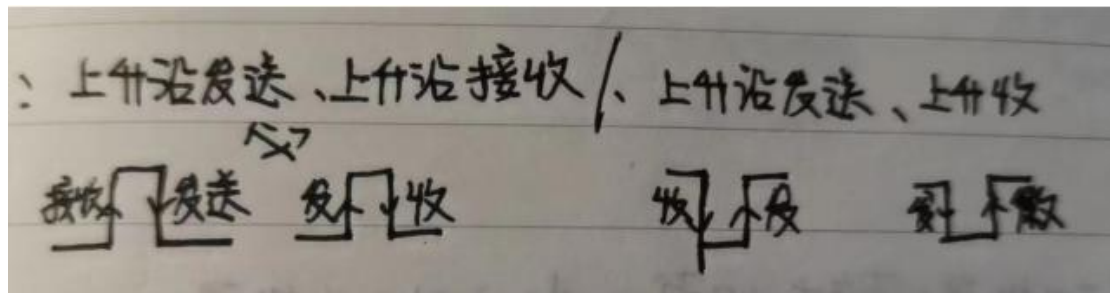
12. 最大传输速率受什么影响?

答：主设备和从设备、时钟频率

13. 四种模式及其对应的时序图

答

:



14. SPI 如何读写？

答： 读操作：

MISO, 接收单片机发来的数据(指令)，发完指令，发 24 位地址。

MOSI, 单片机接收数据。

15. 函数指针和指针函数的区别

答：函数指针是指向函数的指针变量。指针函数是一个返回指针的函数。它是一个函数，其返回类型是一个指针类型。

16. 数组和指针的区别

答：数组是一组连续的内存空间，用于存储相同类型的元素。指针是一个变量，存储着内存地址，可以指向任何类型的数据。

17. 引用和指针相比有什么不用

答：1. 引用必须在声明时进行初始化，并且一旦初始化后，它将永远引用同一个对象。而指针可以被赋予空值(NULL)，表示不指向任何有效的内存地址。

2. 引用一旦被初始化，就不能再引用其他对象了，因此不能被修改为引用另一个对象。指针可以通过重新赋值来指向不同的对象。

3. 引用不需要分配内存，它只是已存在对象的别名。指针需要分配内存来存储指针变量本身，并且可以通过指针动态分配和释放内存。

4. 引用不需要分配内存，它只是已存在对象的别名。指针需要分配内存来存储指针变量本身，并且可以通过指针动态分配和释放内存。

18. 全局变量和局部变量的储存方式

答：全局变量通常存储在静态数据区或全局数据区

局部存储在栈 (stack) 上

19. C 中的内存分配方式有几种

答：1. 静态存储区：静态存储区是在程序运行期间就分配并一直存在的内存区域。它包括存储全局变量和静态变量的内存空间。

静态存储区在程序启动时被分配，直到程序结束才释放。

2. 栈存储区：栈存储区用于管理函数的局部变量和函数调用的上下文信息。在每个函数调用时，会为该函数分配一块栈帧，

其中包含参数、局部变量和返回地址等信息。栈是一种后进先出 (LIFO) 的数据结构，因此栈中的内存块按照函数调用的顺序分配和释放。

3.堆存储区：堆存储区是由程序员手动分配和释放的动态内存区域。它提供了一种灵活的内存分配方式，可以在程序运行时动态地申请和释放内存块。堆存储区通常用于存储动态数据结构，如动态数组、链表和对象等。通过函数如 `malloc()`、`calloc()` 和 `realloc()` 可以从堆上分配内存，并通过 `free()` 函数释放已分配的内存。

4. 常量存储区：常量存储区用于存储常量数据，如字符串常量和全局常量变量。这些数据是在编译时就确定的，并且存储在只读的内存区域中。

20. IIC 通信方式

答：半双工、数据线 `sda`、时钟线 `scl`

21. 中断概念

答：中断是计算机系统中的一种机制，用于处理来自外部设备或内部事件的异步事件

22. 中断过程

答：发生事件->请求中断 ->响应中断 ->中断向量表索引 ->中断服务程序执行 ->结束恢复现场

23. 中断能否传参

答：不能 一般都是通过全局变量方式。不过为了避免中断函数重入，我们可以在进入中断服务函数之后将中断关闭，处理完后再开中断。 中断函数是收到中断信号自动跳转的，并非通过函数调用，什么时候运行是预先不确定的，所以不支持传参数。

24. IIC 数据帧格式

答：start 信号 ->7 位地址和读写位 ->应答位 ->数据 ->应答位 ->stop 信号

25. UART 通信波特率

答：波特率（Baud Rate）是衡量 UART 通信速度的单位，表示每秒传输的比特数。 较低的波特率适用于长距离通信或者噪声环境下，而较高的波特率适用于短距离通信或者要求更高的实时性。

26. UART 有几根线

答：两根传输线 `tx` 和接收线 `rx` `gnd`、`vcc`

27. UART 同步还是异步

答：异步通信

28. UART 数据帧格式

答：起始位、数据位、校验位、停止位

29. IIC 应答数据方式

答：接收方成功接收数据，拉低 **sda** 发送低电平信号，没有接收成功就维持 **sda** 高电平

30. SPI 通信波特率

答：stm32f103 波特率最高是 18mhz

31. ADC 位数精度如何计算

答：一个 12 位的 ADC 可以将输入的模拟信号转换为一个 12 位的数字值，即有 4096 (2^{12}) 个离散的量化水平。

位数精度的计算可以通过以下公式进行：

精度 (in bits) = $\log_2$ (量化水平数)

以 12 位的 ADC 为例，它具有 $2^{12} = 4096$ 个离散的量化水平。因此，它的位数精度可以通过如下公式计算：

精度 (in bits) = $\log_2$ (4096) = 12 bits

对于一个 12 位 ADC，如果其测量范围为 0V 到 3.3V，则最小分辨步进值为：

最小分辨步进值 = $3.3V / (2^{12}) \approx 0.000805V$

32. GPIO 有几种输出模式

答：推挽输出、开漏输出、复用开漏输出、复用推挽输出
一些引脚有 pwm 脉冲宽度调制输出、模拟输出

33. 开漏输出和推挽输出的区别

答：输出电平：

开漏输出：在开漏输出模式下，引脚可以拉低（连接到地/GND），但不能直接提供高电平输出。引脚相当于一个开关，只能将引脚拉低和保持高阻态。要实现高电平，需要外部上拉电阻将引脚拉高到高电平。

推挽输出：在推挽输出模式下，引脚可以提供高电平和低电平输出。当输出高电平时，引脚与高电压连接；当输出低电平时，引脚与地（GND）连接。

输出能力：

开漏输出：开漏输出通常具有较大的输出电流能力，可以驱动较高的负载电流。它经常用于多个引脚共享一个总线/线路的情况，如 I2C 总线。

推挽输出：推挽输出可以提供较大的输出电流。它适用于直接驱动负载的情况，如 LED、电机等。

上拉电阻：

开漏输出：开漏输出模式下，引脚需要外部上拉电阻来将引脚拉高到高电平。这是因为在开漏输出状态下，引脚只负责拉低（连接到地），不直接提供高电平输出。

推挽输出：推挽输出模式下，引脚可以直接提供高电平输出，不需要外部上拉电阻。

多路复用能力：

开漏输出：开漏输出可以用于实现多个引脚共享一个总线/线路的情况，如 I2C 总线。多个开漏输出引脚可以通过外部上拉电阻和总线上的设备进行逻辑连接。

推挽输出：推挽输出引脚通常适合单独驱动独立的设备或信号线

34. 为什么栈从上往下生长？

答：堆是往上涨的，栈是往下涨的，假如栈往上涨的话，那么要确定堆和栈的分界线，每一个程序的堆和栈都一样，所以这个分界线不好确定，所以一个往上涨，一个往下长可以最大化的利用内存的空间。